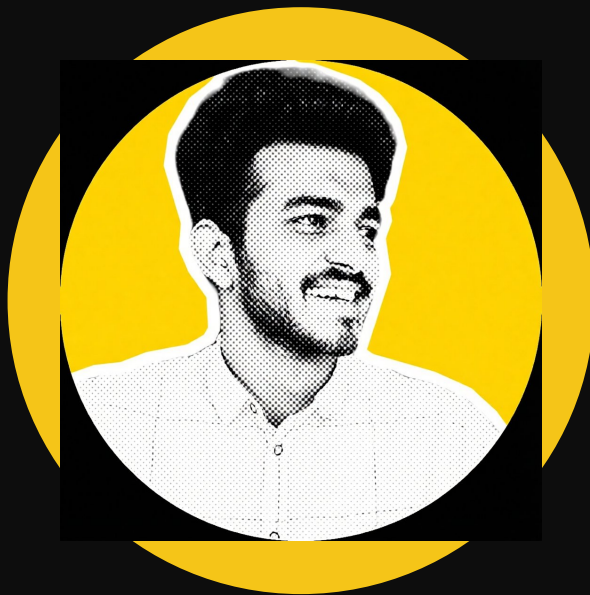




SANS AURA

DSA DOJO

The Ultimate Product-Based Company
Preparation Guide

DSA | LLD | HLD | CS Fundamentals

Freshers

Experienced

All Levels

[instagram.com/sans.aura_](https://www.instagram.com/sans.aura_)

[youtube.com/@Sans_aura](https://www.youtube.com/@Sans_aura)

Version 1.0 | 2025 Edition | Complete Guide

TABLE OF CONTENTS

Your roadmap to cracking product-based companies

01

Product-Based Companies List

Tier-wise company listings for Freshers & Experienced

02

Data Structures & Algorithms (DSA)

Complete topic-wise checklist with 200+ problems

03

Important DSA Questions

Must-do problems for every topic

04

Low Level Design (LLD)

OOP, Design Patterns, SOLID, Machine Coding

05

High Level Design (HLD)

System Design, Scalability, Distributed Systems

06

CS Fundamentals – Operating Systems

Process, Threads, Memory, Deadlocks, Scheduling

07

CS Fundamentals – DBMS

SQL, Normalization, Transactions, Indexing

08

CS Fundamentals – Computer Networks

OSI, TCP/IP, HTTP, DNS, Security

09

CS Fundamentals – OOP Concepts

Pillars of OOP, SOLID Principles, Interview Questions

10

Behavioural & HR Round

Common questions, STAR method, Tips

11

Preparation Strategy & Timeline

Week-wise plan for Freshers & Experienced

Follow us: Instagram: [@sans.aura_](https://www.instagram.com/sans.aura_) | YouTube: [@Sans_aura](https://www.youtube.com/@Sans_aura)

PRODUCT-BASED COMPANIES LIST

Freshers + Experienced

Target companies organized by tier – for Freshers & Experienced professionals

● TIER 1 – Dream / FAANG+ Companies (CTC: 30-80+ LPA)

Extremely competitive. Focus on DSA + System Design + CS Fundamentals.

• Google	• Uber	• Databricks
• Microsoft	• Stripe	• Snowflake
• Amazon	• Airbnb	• Coinbase
• Apple	• LinkedIn	• OpenAI
• Meta (Facebook)	• Twitter/X	• DeepMind
• Netflix	• Palantir	• Anthropic

● TIER 2 – Top Product + Finance Companies (CTC: 18-45+ LPA)

Strong DSA + good LLD/HLD skills. Finance companies need competitive programming skills.

• Adobe	• Oracle	• Rubrik
• Goldman Sachs	• VMware (Broadcom)	• Cloudera
• Morgan Stanley	• Cisco	• Expedia
• Atlassian	• SAP Labs	• Booking.com
• Salesforce	• Qualcomm	• Walmart Global Tech
• Intuit	• Nvidia	• JP Morgan Chase
• Visa	• Intel	• Deutsche Bank
• Mastercard	• ServiceNow	• Barclays
• PayPal	• Splunk	• Tower Research
• Samsung R&D;	• Nutanix	• DE Shaw

● TIER 3 – Indian Unicorns & Top Startups (CTC: 12-35+ LPA)

Great for Freshers! Focus on DSA + LLD. Some need HLD for experienced roles.

• Flipkart	• ShareChat	• BrowserStack
• PhonePe	• Ola	• Postman
• Razorpay	• Dunzo	• Freshworks
• Swiggy	• Navi	• Zoho
• Zomato	• Jupiter	• Chargebee
• Dream11	• Paytm	• Hasura
• CRED	• PolicyBazaar	• Razorpay
• Meesho	• Cars24	• slice
• Groww	• Urban Company	• Jar
• Zerodha	• Delhivery	• INDmoney

● TIER 4 – Service-Based (Product Roles / Elevated Hiring) (CTC: 6-14 LPA)

Good starting point. Focus on DSA basics + CS Fundamentals + Aptitude.

• TCS Digital	• Accenture (Advanced)	• Virtusa
• Infosys (Power Programmer / DSE)	• LTIMindtree	• Mindtree
• Wipro Elite	• Tech Mahindra	• Birlasoft
• Cognizant GenC Elevate	• Hexaware	
• Capgemini (ADAPT)	• Mphasis	

PRO TIP: For **Freshers** – Target Tier 3 & 4 companies first, then work up. For **Experienced (2+ yrs)** – Focus on Tier 1 & 2, prepare System Design heavily. Apply broadly, prepare deeply.

DATA STRUCTURES & ALGORITHMS

Freshers + Experienced

Complete topic-wise checklist – Fresher & Experienced must-know topics

1. Arrays & Strings [Fresher + Experienced]

- Two Pointer Technique
- Sliding Window (Fixed & Variable)
- Prefix Sum & Suffix Arrays
- Kadane's Algorithm (Max Subarray)
- Dutch National Flag (Sort 0,1,2)
- Merge Intervals
- Trapping Rain Water
- Next Permutation
- Rotate Array / Matrix
- String Matching (KMP, Rabin-Karp)
- Longest Substring Without Repeating Chars
- Anagram Checking & Grouping
- Stock Buy & Sell Problems
- Spiral Matrix Traversal
- Subarray Sum Equals K

2. Hashing & HashMaps [Fresher + Experienced]

- Two Sum / Three Sum / Four Sum
- Frequency Counting
- Longest Consecutive Sequence
- Subarray with Given Sum
- Group Anagrams
- Count Distinct Elements in Window
- First Non-Repeating Character
- Longest Substring with K Distinct Chars
- Hashing Collision Resolution (Chaining / Open Addressing)

3. Linked Lists [Fresher + Experienced]

- Reverse a Linked List (Iterative & Recursive)
- Detect & Remove Cycle (Floyd's Algorithm)
- Merge Two Sorted Lists
- Merge K Sorted Lists
- Remove Nth Node from End
- Palindrome Linked List
- Intersection of Two Linked Lists
- Flatten a Multi-Level List
- LRU Cache Implementation
- Clone a List with Random Pointer
- Add Two Numbers (Linked List)
- Rotate Linked List

4. Stacks & Queues [Fresher + Experienced]

- Next Greater / Smaller Element
- Min Stack / Max Stack
- Largest Rectangle in Histogram
- Valid Parentheses
- Implement Queue using Stacks
- Implement Stack using Queues
- Sliding Window Maximum (Deque)
- Stock Span Problem
- Celebrity Problem
- Trapping Rain Water (Stack approach)
- Evaluate Postfix / Prefix Expressions
- Monotonic Stack / Queue Patterns

5. Binary Trees [Fresher + Experienced]

- All Traversals (Inorder, Preorder, Postorder, Level-Order)
- Height / Depth of Tree
- Diameter of Binary Tree
- Check Balanced Tree
- Lowest Common Ancestor (LCA)
- Boundary Traversal
- Vertical Order Traversal

- Top View / Bottom View / Left View / Right View
- Zigzag Level Order Traversal
- Flatten BT to Linked List
- Serialize & Deserialize Binary Tree
- Maximum Path Sum
- Check Mirror / Symmetric Tree
- Construct BT from Inorder + Preorder/Postorder

6. Binary Search Trees (BST) [Fresher + Experienced]

- Search / Insert / Delete in BST
- Validate BST
- Kth Smallest/Largest in BST
- LCA in BST
- Floor & Ceil in BST

- Convert Sorted Array to BST
- Inorder Successor/Predecessor
- Merge Two BSTs
- Two Sum in BST
- Recover BST (Two nodes swapped)

7. Heaps & Priority Queues [Fresher + Experienced]

- Kth Largest/Smallest Element
- Top K Frequent Elements
- Merge K Sorted Lists/Arrays
- Median in a Stream
- Heap Sort

- Task Scheduler
- Reorganize String
- Meeting Rooms I & II
- K Closest Points to Origin
- Min Cost to Connect Sticks

8. Recursion & Backtracking [Fresher + Experienced]

- Subsets / Subsequences Generation
- Permutations (with/without duplicates)
- Combination Sum I, II, III, IV
- N-Queens Problem
- Sudoku Solver
- Word Search / Boggle
- Rat in a Maze
- Generate Parentheses
- Letter Combinations of Phone Number
- Palindrome Partitioning
- Knight's Tour
- M-Coloring Problem

9. Dynamic Programming (DP) [Fresher + Experienced (Critical!)]

- 0/1 Knapsack & Variations
- Unbounded Knapsack
- Longest Common Subsequence (LCS)
- Longest Increasing Subsequence (LIS)
- Edit Distance
- Coin Change (Min Coins / Count Ways)
- Matrix Chain Multiplication
- Partition Equal Subset Sum
- Rod Cutting
- Egg Drop Problem
- Longest Palindromic Subsequence
- Word Break Problem
- Distinct Subsequences
- Burst Balloons
- DP on Trees (House Robber III)
- DP on Grids (Unique Paths, Min Path Sum)
- DP with Bitmask
- Digit DP (Experienced)
- String DP (Wildcard, Regex Matching)
- Stock Problems (All variants)

10. Graphs [Fresher + Experienced (Very Important!)]

- BFS / DFS Traversal
- Number of Islands / Connected Components
- Cycle Detection (Directed & Undirected)
- Topological Sort (Kahn's + DFS)
- Shortest Path: Dijkstra's Algorithm
- Shortest Path: Bellman-Ford
- Shortest Path: Floyd-Warshall
- Minimum Spanning Tree (Prim's & Kruskal's)
- Union-Find / Disjoint Set Union (DSU)
- Bipartite Graph Check
- Bridges & Articulation Points (Tarjan's)
- Strongly Connected Components (Kosaraju's)
- Word Ladder I & II
- Clone Graph
- Alien Dictionary
- Course Schedule I & II
- Cheapest Flights Within K Stops
- Network Delay Time
- Accounts Merge
- Graph Coloring

11. Binary Search (Advanced) [Fresher + Experienced]

- Search in Rotated Sorted Array
- Find Minimum in Rotated Array
- Median of Two Sorted Arrays
- Koko Eating Bananas
- Aggressive Cows / Book Allocation
- Painter's Partition
- Peak Element
- Search in 2D Matrix
- Binary Search on Answer (Pattern)
- Split Array Largest Sum
- Capacity to Ship Packages
- Nth Root of a Number

12. Greedy Algorithms [Fresher + Experienced]

- Activity Selection / Job Scheduling
- Fractional Knapsack
- Huffman Encoding
- Minimum Platforms
- Jump Game I & II
- Gas Station
- Candy Distribution
- Assign Cookies
- Non-overlapping Intervals
- Meeting Rooms
- Minimum Number of Coins

13. Tries [Experienced (Fresher: Basics)]

- Implement Trie (Insert, Search, StartsWith)
- Word Search II (Trie + Backtracking)
- Longest Common Prefix using Trie
- Auto-Complete System
- Maximum XOR of Two Numbers (Trie)
- Count Distinct Substrings
- Palindrome Pairs

14. Segment Trees & BIT [Experienced (Competitive)]

- Range Sum Query (Mutable)
- Range Minimum Query
- Lazy Propagation
- Count of Smaller Numbers After Self
- Binary Indexed Tree (Fenwick Tree)
- Merge Sort Tree
- Interval Scheduling with Updates

15. Bit Manipulation [Fresher + Experienced]

- Single Number (I, II, III)
- Power of Two / Four
- Counting Bits
- Subsets using Bitmask
- XOR from 1 to N
- Find Two Non-Repeating Elements
- Reverse Bits
- Missing Number (XOR approach)

MUST-DO DSA QUESTIONS (TOP 100+)

These questions cover 90%+ of what product-based companies ask

Freshers + Experienced

Arrays & Strings (15 Must-Do)

1. Two Sum	9. Longest Substring Without Repeating Characters
2. Best Time to Buy & Sell Stock	10. Next Permutation
3. Maximum Subarray (Kadane's)	11. Rotate Image
4. Merge Intervals	12. Set Matrix Zeroes
5. 3Sum	13. Spiral Matrix
6. Container With Most Water	14. 4Sum
7. Trapping Rain Water	15. Longest Palindromic Substring
8. Product of Array Except Self	

Linked List (10 Must-Do)

1. Reverse Linked List	6. Intersection of Two Linked Lists
2. Merge Two Sorted Lists	7. Palindrome Linked List
3. Linked List Cycle (I & II)	8. Copy List with Random Pointer
4. Remove Nth Node From End	9. LRU Cache
5. Add Two Numbers	10. Merge K Sorted Lists

Stack & Queue (8 Must-Do)

1. Valid Parentheses	5. Sliding Window Maximum
2. Min Stack	6. Evaluate Reverse Polish Notation
3. Next Greater Element	7. Daily Temperatures
4. Largest Rectangle in Histogram	8. Implement Queue using Stacks

Trees (12 Must-Do)

1. Maximum Depth of Binary Tree	7. Serialize & Deserialize BT
2. Invert Binary Tree	8. Binary Tree Maximum Path Sum
3. Validate BST	9. Kth Smallest in BST
4. Lowest Common Ancestor	10. Construct BT from Preorder & Inorder
5. Binary Tree Level Order Traversal	11. Vertical Order Traversal
6. Diameter of Binary Tree	12. Boundary Traversal

Graphs (12 Must-Do)

1. Number of Islands	7. Detect Cycle (Directed + Undirected)
2. Clone Graph	8. Minimum Spanning Tree
3. Course Schedule (I & II)	9. Accounts Merge
4. Word Ladder	10. Alien Dictionary
5. Dijkstra's Algorithm	11. Cheapest Flights Within K Stops
6. Topological Sort	12. Network Delay Time

Dynamic Programming (15 Must-Do)

1. Climbing Stairs	9. Unique Paths
2. Coin Change	10. Partition Equal Subset Sum
3. Longest Common Subsequence	11. Palindrome Partitioning
4. Longest Increasing Subsequence	12. Matrix Chain Multiplication
5. 0/1 Knapsack	13. Burst Balloons
6. Edit Distance	14. Distinct Subsequences
7. Word Break	15. Minimum Path Sum
8. House Robber (I, II, III)	

Binary Search (8 Must-Do)

1. Search in Rotated Sorted Array	5. Aggressive Cows / Book Allocation
2. Find Minimum in Rotated Array	6. Peak Element
3. Median of Two Sorted Arrays	7. Search a 2D Matrix
4. Koko Eating Bananas	8. Split Array Largest Sum

Greedy (6 Must-Do)

1. Activity Selection	4. Non-overlapping Intervals
2. Jump Game (I & II)	5. Meeting Rooms (I & II)
3. Gas Station	6. Candy Distribution

Backtracking (6 Must-Do)

1. N-Queens	4. Permutations (I & II)
2. Sudoku Solver	5. Combination Sum (I & II)
3. Subsets (I & II)	6. Word Search

Bit Manipulation (5 Must-Do)

1. Single Number (I, II, III)	4. Reverse Bits
2. Power of Two	5. Missing Number
3. Counting Bits	

LOW LEVEL DESIGN (LLD)

Object-Oriented Design, Design Patterns, SOLID, Machine Coding Rounds

Freshers + Experienced

A. Object-Oriented Programming (OOP) Core

- **Classes & Objects:** Constructors, destructors, access modifiers, static vs instance
- **4 Pillars of OOP:** Encapsulation, Abstraction, Inheritance, Polymorphism
- **Inheritance Types:** Single, Multilevel, Hierarchical, Hybrid; Diamond Problem in C++/Java
- **Polymorphism:** Compile-time (Overloading) vs Runtime (Overriding); Virtual functions, vtable
- **Abstraction:** Abstract classes vs Interfaces; When to use which
- **Encapsulation:** Getters/Setters, why private fields, immutability
- **Composition vs Inheritance:** Favor composition; Has-A vs Is-A relationship
- **Association, Aggregation, Composition:** UML relationships; Lifetime dependency differences

B. SOLID Principles [Asked in every LLD round]

- **S – Single Responsibility:** A class should have only one reason to change
- **O – Open/Closed:** Open for extension, closed for modification
- **L – Liskov Substitution:** Subtypes must be substitutable for their base types
- **I – Interface Segregation:** Many small interfaces better than one fat interface
- **D – Dependency Inversion:** Depend on abstractions, not concretions

C. Design Patterns [Experienced: Must Know | Fresher: Basics]

Creational Patterns: Singleton (Thread-safe), Factory Method, Abstract Factory, Builder Pattern, Prototype Pattern

Structural Patterns: Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight

Behavioral Patterns: Strategy, Observer, Command, State, Template Method, Iterator, Chain of Responsibility, Mediator

D. Machine Coding Round Problems [Most Important for LLD Interviews]

1. Design Parking Lot System	13. Design File System (In-Memory)
2. Design Elevator System	14. Design Library Management System
3. Design Snake & Ladder Game	15. Design Hotel Booking System
4. Design Tic-Tac-Toe	16. Design ATM Machine
5. Design Chess Game	17. Design Vending Machine
6. Design BookMyShow / Movie Ticket Booking	18. Design Ride-Sharing App (Uber/Ola)
7. Design Splitwise (Expense Sharing)	19. Design Online Coding Judge
8. Design Food Delivery System (Swiggy/Zomato)	20. Design Inventory Management System
9. Design Logging Framework	21. Design Car Rental System
10. Design Cache (LRU / LFU)	22. Design Meeting Scheduler
11. Design Pub-Sub Messaging System	23. Design Amazon Locker System
12. Design Rate Limiter	24. Design Stack Overflow

HIGH LEVEL DESIGN (HLD) / SYSTEM DESIGN

Experienced (Freshers: Basics)

Architecture, Scalability, Distributed Systems – Critical for Experienced roles

A. Core System Design Concepts

- **Scalability:** Horizontal vs Vertical scaling; Auto-scaling; Elasticity
- **Load Balancing:** Round Robin, Least Connections, Consistent Hashing; L4 vs L7; HAProxy, Nginx
- **Caching:** Cache-aside, Write-through, Write-back; Redis, Memcached; Cache eviction (LRU, LFU, TTL)
- **Database Design:** SQL vs NoSQL; Sharding (Horizontal/Vertical); Replication (Master-Slave, Master-Master)
- **CAP Theorem:** Consistency, Availability, Partition Tolerance – pick two; CP vs AP systems
- **Consistent Hashing:** Virtual nodes, ring-based distribution; Used in DynamoDB, Cassandra
- **Message Queues:** Kafka, RabbitMQ, SQS; Pub-Sub vs Point-to-Point; Event-driven architecture
- **Microservices:** Service decomposition, API Gateway, Service Mesh, Circuit Breaker
- **API Design:** REST vs GraphQL vs gRPC; Pagination, Rate Limiting, Versioning, Authentication
- **CDN:** Content Delivery Networks; Pull vs Push CDN; Edge caching
- **Databases Deep Dive:** ACID properties; Indexing (B-Tree, Hash, Composite); Transactions & Isolation levels
- **Consensus Algorithms:** Raft, Paxos; Leader election; Distributed consensus
- **Data Storage:** Blob storage (S3), Time-series DB, Column-store, Document DB, Graph DB
- **Monitoring & Logging:** ELK Stack, Prometheus, Grafana; Distributed tracing; Health checks
- **Security:** OAuth 2.0, JWT, HTTPS, Encryption at rest/transit; Rate limiting, DDoS protection

B. System Design Interview Questions [TOP 25 Most Asked]

1. Design URL Shortener (TinyURL)	14. Design Rate Limiter
2. Design Twitter / X	15. Design Web Crawler
3. Design Instagram / Photo Sharing	16. Design Pastebin
4. Design WhatsApp / Chat System	17. Design News Feed (Facebook)
5. Design YouTube / Video Streaming	18. Design Ticketmaster / BookMyShow
6. Design Netflix	19. Design Typeahead / Autocomplete
7. Design Uber / Ride-Sharing Service	20. Design Distributed Cache
8. Design Zomato / Food Delivery	21. Design Payment System (Razorpay)
9. Design Amazon / E-Commerce Platform	22. Design Chat Application (Slack)
10. Design Google Search	23. Design Logging & Monitoring System
11. Design Google Maps / Navigation	24. Design Search Autocomplete
12. Design Dropbox / Google Drive	25. Design Content Delivery Network (CDN)
13. Design Notification System	

SYSTEM DESIGN APPROACH (STEPS):

1. Clarify Requirements (Functional + Non-Functional) →
2. Estimate Scale (Users, QPS, Storage) →
3. High-Level Architecture (Draw components) →
4. Deep Dive into Core Components →
5. Address Bottlenecks & Trade-offs →
6. Discuss Monitoring, Alerts, Failure handling

CS FUNDAMENTALS – OPERATING SYSTEMS

Freshers + Experienced

Core OS concepts asked in every product-based company interview

Process Management

- Process vs Thread – differences, similarities, context switching cost
- Process states – New, Ready, Running, Waiting, Terminated
- Process Control Block (PCB) – what it stores
- Inter-Process Communication (IPC) – Shared Memory, Message Passing, Pipes, Sockets
- fork() system call – how child processes are created
- Zombie Process vs Orphan Process
- Multithreading models – Many-to-One, One-to-One, Many-to-Many
- User-level threads vs Kernel-level threads

CPU Scheduling

- FCFS (First Come First Served)
- SJF (Shortest Job First) – Preemptive & Non-Preemptive
- Round Robin – time quantum, context switching overhead
- Priority Scheduling – starvation, aging solution
- Multilevel Queue & Multilevel Feedback Queue
- Convoy Effect in FCFS
- Compare all algorithms: throughput, turnaround time, waiting time

Synchronization & Deadlocks

- Critical Section Problem – Mutual Exclusion, Progress, Bounded Waiting
- Mutex vs Semaphore (Binary & Counting) – differences and usage
- Producer-Consumer Problem
- Readers-Writers Problem
- Dining Philosophers Problem
- Deadlock – necessary conditions (Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait)
- Deadlock Prevention, Avoidance (Banker's Algorithm), Detection, Recovery
- Starvation vs Deadlock
- Priority Inversion & Priority Inheritance

Memory Management

- Contiguous vs Non-Contiguous allocation
- Paging – page table, page size, internal fragmentation
- Segmentation – segment table, external fragmentation
- Virtual Memory – page fault, demand paging, page replacement
- Page Replacement Algorithms – FIFO, LRU, Optimal, Clock
- Thrashing – causes & solutions (working set model)
- Translation Lookaside Buffer (TLB) – purpose, hit/miss
- Memory Mapped I/O
- Logical vs Physical Address Space

File Systems & Storage

- File allocation methods – Contiguous, Linked, Indexed
- Directory structures – Single-level, Two-level, Tree
- Disk Scheduling – FCFS, SSTF, SCAN, C-SCAN, LOOK
- RAID Levels – 0, 1, 5, 6, 10 – trade-offs
- Inode structure – what it stores

Top OS Interview Questions [Frequently Asked]

1. What is the difference between process and thread?
2. Explain context switching. What is its overhead?
3. What are the different states of a process?
4. Explain deadlock. What are the necessary conditions?
5. Difference between mutex and semaphore?
6. What is virtual memory? How does paging work?
7. Explain page replacement algorithms (LRU, FIFO, Optimal).
8. What is thrashing? How to prevent it?
9. Difference between preemptive and non-preemptive scheduling?
10. What is Belady's Anomaly?
11. Explain Banker's Algorithm for deadlock avoidance.
12. What is a critical section? How to solve it?
13. Difference between internal and external fragmentation?
14. What is TLB? What happens on a TLB miss?
15. Explain the Producer-Consumer problem with semaphores.

CS FUNDAMENTALS – DBMS

Freshers + Experienced

Database Management Systems – SQL, Normalization, Transactions, Indexing

Core DBMS Concepts

- DBMS vs RDBMS vs NoSQL – when to use what
- Schema, Instance, Data Independence (Logical & Physical)
- Three-Schema Architecture (External, Conceptual, Internal)
- ER Model – Entity, Attributes, Relationships, Cardinality
- Relational Model – Tuples, Attributes, Domain, Keys
- Keys – Primary, Candidate, Super, Foreign, Composite, Alternate

Normalization

- 1NF – Eliminate repeating groups, atomic values
- 2NF – Remove partial dependencies (on part of composite key)
- 3NF – Remove transitive dependencies
- BCNF (Boyce-Codd NF) – Every determinant is a candidate key
- Denormalization – when and why to denormalize (performance vs redundancy)
- Functional Dependencies, Closure, Canonical Cover

SQL – Must Know Queries

- JOINS – INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF JOIN
- Subqueries – Scalar, Row, Table, Correlated subqueries
- GROUP BY, HAVING, ORDER BY, DISTINCT
- Window Functions – ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, NTILE
- Aggregate Functions – COUNT, SUM, AVG, MIN, MAX
- UNION vs UNION ALL, INTERSECT, EXCEPT
- Views – Simple vs Complex, Materialized Views
- Stored Procedures, Triggers, Cursors
- Find Nth highest salary (multiple approaches)
- Find duplicate records, delete duplicates
- Employees earning more than their manager
- Department-wise highest salary

Transactions & Concurrency

- ACID Properties – Atomicity, Consistency, Isolation, Durability
- Transaction States – Active, Partially Committed, Committed, Failed, Aborted
- Concurrency Problems – Dirty Read, Non-Repeatable Read, Phantom Read, Lost Update
- Isolation Levels – Read Uncommitted, Read Committed, Repeatable Read, Serializable
- Lock-Based Protocols – Shared Lock, Exclusive Lock, 2-Phase Locking (2PL)
- Deadlock in DBMS – Detection (Wait-For Graph), Prevention, Timeouts
- MVCC (Multi-Version Concurrency Control) – how PostgreSQL/MySQL implement it

Indexing & Storage

- B-Tree vs B+ Tree – structure, advantages for range queries
- Hash Index – for equality lookups
- Clustered vs Non-Clustered Index
- Composite Index – column order matters
- Covering Index – index-only scan
- When to create / avoid indexes
- Query Optimization – Execution plans, EXPLAIN ANALYZE

NoSQL Concepts

- Types: Key-Value (Redis), Document (MongoDB), Column-Family (Cassandra), Graph (Neo4j)
- CAP Theorem applied to NoSQL databases
- Eventual Consistency vs Strong Consistency
- Sharding strategies, Replication, Partitioning
- When to choose SQL vs NoSQL

Top DBMS Interview Questions [Frequently Asked]

1. What are ACID properties? Explain each.
2. Difference between SQL and NoSQL?
3. Explain normalization up to BCNF with examples.
4. What is indexing? Types of indexes?
5. Write a query to find the 2nd highest salary.
6. Explain different types of JOINS with examples.
7. What are window functions? When to use them?
8. Explain the difference between DELETE, TRUNCATE, and DROP.
9. What is a deadlock in DBMS? How to handle it?
10. Explain different isolation levels.
11. What is a B+ Tree? Why is it used for indexing?
12. Difference between Clustered and Non-Clustered Index?
13. What is MVCC? How does it help concurrency?
14. When would you choose NoSQL over SQL?
15. Explain sharding. What are its challenges?

CS FUNDAMENTALS – COMPUTER NETWORKS

Freshers + Experienced

OSI Model, TCP/IP, HTTP, DNS, Security – frequently asked in interviews

OSI & TCP/IP Models

- OSI 7 Layers – Physical, Data Link, Network, Transport, Session, Presentation, Application
- TCP/IP 4 Layers – Network Access, Internet, Transport, Application
- Protocols at each layer – Ethernet, IP, TCP, UDP, HTTP, FTP, SMTP, DNS
- Encapsulation & Decapsulation process

Transport Layer

- TCP vs UDP – connection-oriented vs connectionless, reliability, speed
- TCP 3-Way Handshake (SYN, SYN-ACK, ACK) & 4-Way Termination
- TCP Flow Control – Sliding Window Protocol
- TCP Congestion Control – Slow Start, AIMD, Fast Retransmit, Fast Recovery
- Ports – Well-known (0-1023), Registered (1024-49151), Dynamic (49152-65535)
- Socket Programming – how client-server communication works

Network Layer

- IPv4 vs IPv6 – address space, header format, NAT elimination
- Subnetting & CIDR – calculating network address, broadcast address, usable hosts
- Routing – Static vs Dynamic; Distance Vector vs Link State
- Routing Protocols – RIP, OSPF, BGP
- NAT (Network Address Translation) – types, how it works
- ARP (Address Resolution Protocol) – MAC to IP resolution
- ICMP – ping, traceroute

Application Layer Protocols

- HTTP/HTTPS – methods (GET, POST, PUT, DELETE), status codes, headers
- HTTP/1.1 vs HTTP/2 vs HTTP/3 – multiplexing, server push, QUIC
- DNS – recursive vs iterative queries, DNS hierarchy, caching, record types (A, AAAA, CNAME, MX)
- DHCP – how IP addresses are assigned automatically
- FTP, SMTP, POP3, IMAP – email and file transfer protocols
- WebSockets – full-duplex communication, use cases (chat, real-time)
- REST vs GraphQL vs gRPC – comparison, when to use

Security

- Symmetric vs Asymmetric Encryption – AES, RSA, Diffie-Hellman
- SSL/TLS Handshake – how HTTPS works, certificates
- Firewalls – packet filtering, stateful inspection, proxy
- VPN – tunneling protocols, IPsec
- Common Attacks – DDoS, Man-in-the-Middle, DNS Spoofing, SQL Injection, XSS, CSRF
- OAuth 2.0, JWT – token-based authentication flow
- CORS – Cross-Origin Resource Sharing, same-origin policy

Top Computer Networks Interview Questions [Frequently Asked]

1. Explain the OSI model. What happens at each layer?

2. Difference between TCP and UDP? When to use each?
3. Explain the TCP 3-way handshake.
4. What happens when you type google.com in the browser?
5. What is DNS? How does DNS resolution work?
6. Difference between HTTP and HTTPS?
7. What is a subnet mask? How to calculate subnets?
8. Explain how HTTPS/TLS handshake works.
9. What is a CDN? How does it improve performance?
10. Difference between IPv4 and IPv6?
11. What is ARP? How does it work?
12. Explain WebSockets. How are they different from HTTP?
13. What is the difference between HTTP/1.1, HTTP/2, and HTTP/3?
14. What is NAT? Why is it needed?
15. Explain common security attacks and their prevention.

CS FUNDAMENTALS – OOP CONCEPTS

Freshers + Experienced

Object-Oriented Programming – Deep dive for interview preparation

4 Pillars – Deep Dive

- Encapsulation – Data hiding, access modifiers (private, protected, public, default/package)
- Abstraction – Abstract class vs Interface; Abstract class can have constructors, interfaces cannot (Java)
- Inheritance – Code reuse; Types: Single, Multilevel, Hierarchical, Multiple (via interfaces in Java)
- Polymorphism – Compile-time (Method Overloading) vs Runtime (Method Overriding); Dynamic dispatch

Advanced OOP Concepts

- Constructors – Default, Parameterized, Copy; Constructor chaining; super() vs this()
- Static keyword – static variables, methods, blocks, nested classes; memory area
- Final keyword – final variable, method, class; effectively final in lambdas
- Abstract vs Interface (Java 8+) – default methods, static methods, functional interfaces
- Diamond Problem – Multiple inheritance issue; how Java & C++ handle it
- Shallow Copy vs Deep Copy – Cloneable interface, clone() method
- Immutable classes – how to create (String is immutable – why?)
- Object class methods – equals(), hashCode(), toString(), finalize(), wait(), notify()
- Exception Handling – Checked vs Unchecked, try-catch-finally, try-with-resources, custom exceptions
- Generics – Type safety, bounded types, wildcards (? extends, ? super)
- Collections Framework – List, Set, Map, Queue; HashMap internals (bucket, hashing, collision)
- Comparable vs Comparator – natural ordering vs custom sorting
- Lambda Expressions & Functional Interfaces – Predicate, Function, Consumer, Supplier
- Streams API – filter, map, reduce, collect; lazy evaluation

Top OOP Interview Questions **[Frequently Asked]**

1. What are the 4 pillars of OOP? Explain each with examples.
2. Difference between Abstract Class and Interface?
3. What is the Diamond Problem? How does Java solve it?
4. Explain method overloading vs overriding.
5. What is the SOLID principle? Explain each with examples.
6. What are Design Patterns? Name a few you've used.
7. Explain the Singleton pattern. Make it thread-safe.
8. What is the difference between Composition and Inheritance?
9. How does HashMap work internally in Java?
10. What is the difference between == and equals() in Java?
11. Explain Exception hierarchy in Java.
12. What are Generics? Why are they useful?
13. Explain shallow copy vs deep copy.
14. What is an immutable class? How to create one?
15. What is polymorphism? Give a real-world example.

BEHAVIOURAL & HR ROUND

Freshers + Experienced

Common questions, STAR method, and tips for acing the HR round

A. The STAR Method (Use this framework for EVERY answer)

- **S – Situation:** Set the context. Describe the scenario briefly.
- **T – Task:** What was your responsibility or role?
- **A – Action:** What specific actions did YOU take?
- **R – Result:** What was the outcome? Quantify if possible.

B. Common Behavioural Questions

1. Tell me about yourself. (Prepare a 2-min pitch!)
2. Why do you want to work at [Company]?
3. What is your biggest strength / weakness?
4. Tell me about a challenging project you worked on.
5. Describe a time you had a conflict with a team member. How did you resolve it?
6. Tell me about a time you failed. What did you learn?
7. How do you prioritize multiple tasks with tight deadlines?
8. Describe a time you went above and beyond.
9. Where do you see yourself in 5 years?
10. Why are you leaving your current company? (Experienced)
11. What is your expected CTC / salary?
12. Tell me about a time you disagreed with your manager.
13. How do you stay updated with new technologies?
14. Describe a time you had to learn something quickly.
15. What questions do you have for us? (Always ask 2-3!)

C. HR Round Tips

- Research the company thoroughly before the interview – products, culture, recent news.
- Prepare 3-4 STAR stories covering: leadership, conflict, failure, achievement.
- Be authentic – don't memorize answers, understand the essence.
- Ask thoughtful questions: team structure, tech stack, growth opportunities.
- For salary negotiation: research market rates, be confident, never give a range first.
- Body language matters: eye contact, confident posture, smile.
- For Freshers: highlight projects, internships, competitive programming achievements.
- For Experienced: quantify impact – 'Reduced latency by 40%' > 'Improved performance'.

PREPARATION STRATEGY & TIMELINE

Week-by-week plan for Freshers and Experienced professionals

Freshers + Experienced

● FOR FRESHERS (3-Month Plan)

Week 1-2	Basics	Arrays, Strings, Sorting algorithms, Time/Space complexity (Big O)
Week 3-4	Core DSA	Linked Lists, Stacks, Queues, Hashing, Binary Search
Week 5-6	Trees & Graphs	Binary Trees, BST, BFS/DFS, Graph algorithms
Week 7-8	DP & Backtracking	1D DP, 2D DP, Knapsack, Backtracking problems
Week 9	Advanced DSA	Heaps, Tries, Greedy, Bit Manipulation
Week 10	CS Fundamentals	OS, DBMS, CN, OOP – cover all key topics
Week 11	LLD Basics	OOP design, SOLID, 3-4 Machine Coding problems
Week 12	Mock & Revision	Mock interviews, revise weak topics, previous year questions

● FOR EXPERIENCED (2-Month Intensive Plan)

Week 1-2	DSA Revision	Revise all core DSA topics; focus on medium-hard problems on LeetCode
Week 3-4	System Design (HLD)	Learn core concepts; practice 8-10 system design problems
Week 5	LLD Deep Dive	Design Patterns, SOLID, 5-6 Machine Coding problems
Week 6	CS Fundamentals	Quick revision of OS, DBMS, CN; focus on trade-offs & real-world
Week 7	Company-Specific Prep	Previous interview questions, mock system design rounds
Week 8	Mock & Polish	Full mock interviews, behavioral prep, salary negotiation prep

Recommended Daily Routine

- 2-3 DSA problems daily (Easy → Medium → Hard progression)
- 1 CS Fundamental topic (30-45 mins reading/revision)
- 1 System Design concept or problem (for experienced, 45-60 mins)
- Weekend: 1 Machine Coding round practice (2-hour timer)
- Track progress – maintain a spreadsheet / Notion board of solved problems
- Practice on: LeetCode, GeeksforGeeks, Codeforces, InterviewBit



SANS AURA DSA DOJO

Your journey to cracking product-based companies starts here.

FOLLOW US FOR MORE CONTENT

Instagram: @sans.aura_
[instagram.com/sans.aura_/reels/](https://www.instagram.com/sans.aura_/reels/)

YouTube: @Sans_aura
[youtube.com/@Sans_aura](https://www.youtube.com/@Sans_aura)

"Consistency beats talent when talent doesn't work hard."